THE UNIVERSITY of EDINBURGH

Algebra Geometry & Quantum Fields

Lecture 8: Scientific computing with julia

Job Feldbrugge

AGQ Computing Course

20 March 2026

Scientific computing

- Scientific computing is integral to many branches of research both in mathematics and in physics

- Different languages suite different tasks

Symbolic

Wolfram Mathematica

Maple™

MAPLE

Prototyping

python™

MATLAB®

R

High Performance Computing

C++

Fortran
Programming language

julia

Scientific computing

- Julia is a new language (first released on February 14, 2012)
- Julia is aimed at solving the “two-language problem”, serving both as a proto-typing language (like Python) and a High Performance Computing language (like C++)
- Over the last decade, Julia has grown into an extremely powerful open source language that is greatly suited to scientific research
- Julia is now being used outside of academia (Quantum Computing)
- I moved from Mathematica + Python + c++ to Mathematica + Julia

What makes a language fast?

- Programming languages balance different goals
  - **Readablity and ease of coding v.s. being explicit.** How precise to you code? Do you need to specity all variable types?
  - **Interpreted v.s. compiled languages.** Is your code directly interpreted, or is the code first compiled to an executable?
  - **Checking for types.** While running, does your code check whether operations are allowed?

What makes a language fast?

- Julia is **readable** (like an interpreted language) but is **compiled** (like C++) using the **Just-In-Time (JIT) framework.** By default, do not specify types!

- Contrary to Python and C++, Julia is not an **object-oriented** but a **function-oriented language**

- A function is interpreted as a **recipe** and is only compiled into an **executable** when called with a given set of arguments. Using abstraction, a single function can lead to many executables! **One function, multiple instances.**

- Julia compiled code **minimises checks** in **favour of speed**

Object- v.s. function-oriented

python™

```python
class Student:
    # The constructor method to initialize new objects
    def __init__(self, name, age):
        self.name = name  # Attribute for the student's name
        self.age = age    # Attribute for the student's age

    # A method (a function inside the class)
    def codes(self):
        return f"{self.name} writes code!"

# Create instances (objects) from the class
student_1 = Student("Alice", 20)
student_2 = Student("Lucy", 25)

# Access attributes and call methods of each object
print(f"{student_1.name} is {student_1.age} years old.")
print(student_2.codes())
```

✓ 0.0s

```
Alice is 20 years old.
Lucy writes code!
```

julia

```julia
struct Student
    name
    age
end

function codes(student::Student)
    println(student.name, " writes code!")
end

# Create instances of the structs
student_1 = Student("Alice", 20)
student_2 = Student("Lucy", 25)

# Access attributes and call functions of each object
println(student_1.name, " is ", student_1.age, " years old.")
codes(student_2)
```

✓ 0.0s

```
Alice is 20 years old.
Lucy writes code!
```

Multiple instances

- Always write your code in functions to benefit from the speed of Julia
- A single function can lead to multiple instances (executables)
- The first time a function runs, it is slow. Subsequent runs are fast.
- Do not specify too many types!

```julia
# Slow: uncompiled
println("Slow:\t\t ", 5 + 2)

# Fast: compiled
function add(a, b)
    return a + b
end

# Instance with integers
println("Integers:\t ", add(5, 2))

# Instance with floats
println("Floats:\t\t ", add(5., 2.))

# Instance with complex numbers
println("Complex Floats\t ", add(5. + 2im, 2. + 5im))
```

✓ 0.0s

```
Slow:           7
Integers:       7
Floats:         7.0
Complex Floats  7.0 + 7.0im
```

Two-language problem

- You may say: “But Python is also fast using vectorisation, numpy, scipy and for example PyTorch”

- True, but these libraries outsource the computation to precompiled functions in C++. C++ is fast, not Python!

```python
import math

for i in range(1, 5):
    print(math.factorial(i))
```

✓ 0.0s

```
1
2
6
24
```

```julia
function tmp()
    for i in 1:4
        println(factorial(i))
    end
end

tmp()
```

✓ 0.0s

```
1
2
6
24
```

Broadcasting

A key feature of the Julia language is broadcasting, applying a function to a list, matrix or array

By adding a “.”, you apply a function pointwise to each element of the array.

```julia
list = [1, 5, 3, 2]
factorial.(list)
```

✓ 0.0s

```
4-element Vector{Int64}:
   1
 120
   6
   2
```

Parallelisation

- Easy parallelisation for for-loops.

Sequential

```julia
function tmp()
    for i in 1:4
        println(factorial(i))
    end
end

tmp()
```

0.0s

```
1
2
6
24
```

Parallel

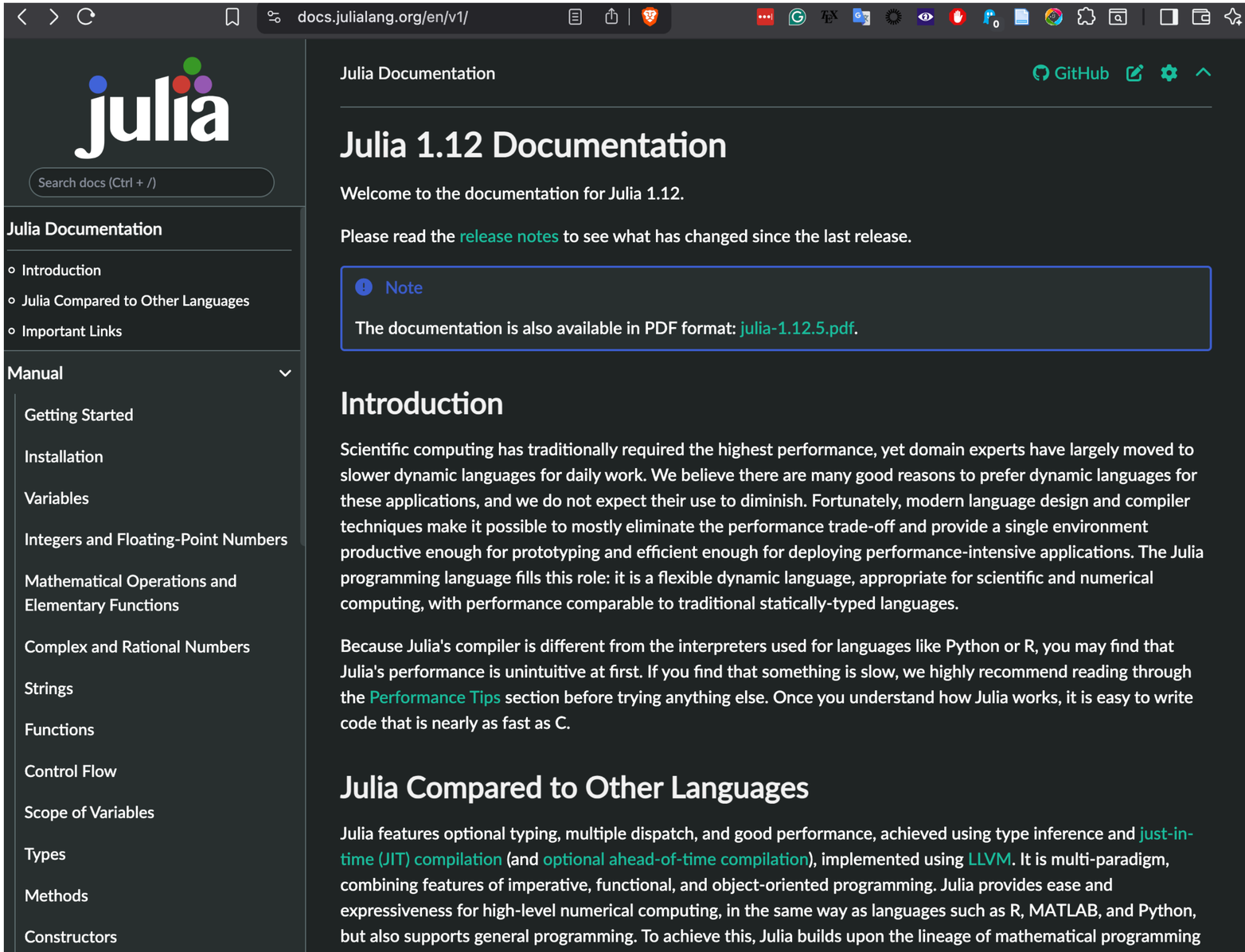
```julia
using Base.Threads

function tmp_parallel()
    @threads  for i in 1:4
        println(factorial(i))
    end
end

tmp_parallel()
```

0.2s

```
6
24
1
2
```

Documentation

- Great documentation at docs.julialang.org/en/v1/
- MIT course by Chris Rackauckas, “Parallel Computing and Scientific Machine learning” on youtube with book: https://book.sciml.ai/
- Many tutorials available



Installing and using Julia

Installation

I recommend using JuliaUp from julialang.org/downloads

Linux and Mac

Installing Julia

It appears that you are using macOS or Linux. Install Julia by running the following in your terminal. In case you are using Windows, please follow these instructions for Windows.

```
$ curl -fsSL https://install.julialang.org | sh
```

COPY

This will install the latest stable version of Julia, as well as the `juliaup` tool. Start Julia from the command-line by typing `julia`. See `juliaup --help` for how to configure installed versions.

If you prefer to use manual installation using a GUI-based installer, see the Manual Downloads page.

Windows

Installing Julia

It appears that you are using Windows. Install Julia using the MSIX App Installer. Alternatively, if you have access to the Microsoft Store, you can install Julia by running the following in the command prompt. In case you are not using Windows, please follow the Linux and macOS instructions.

```
> winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

COPY

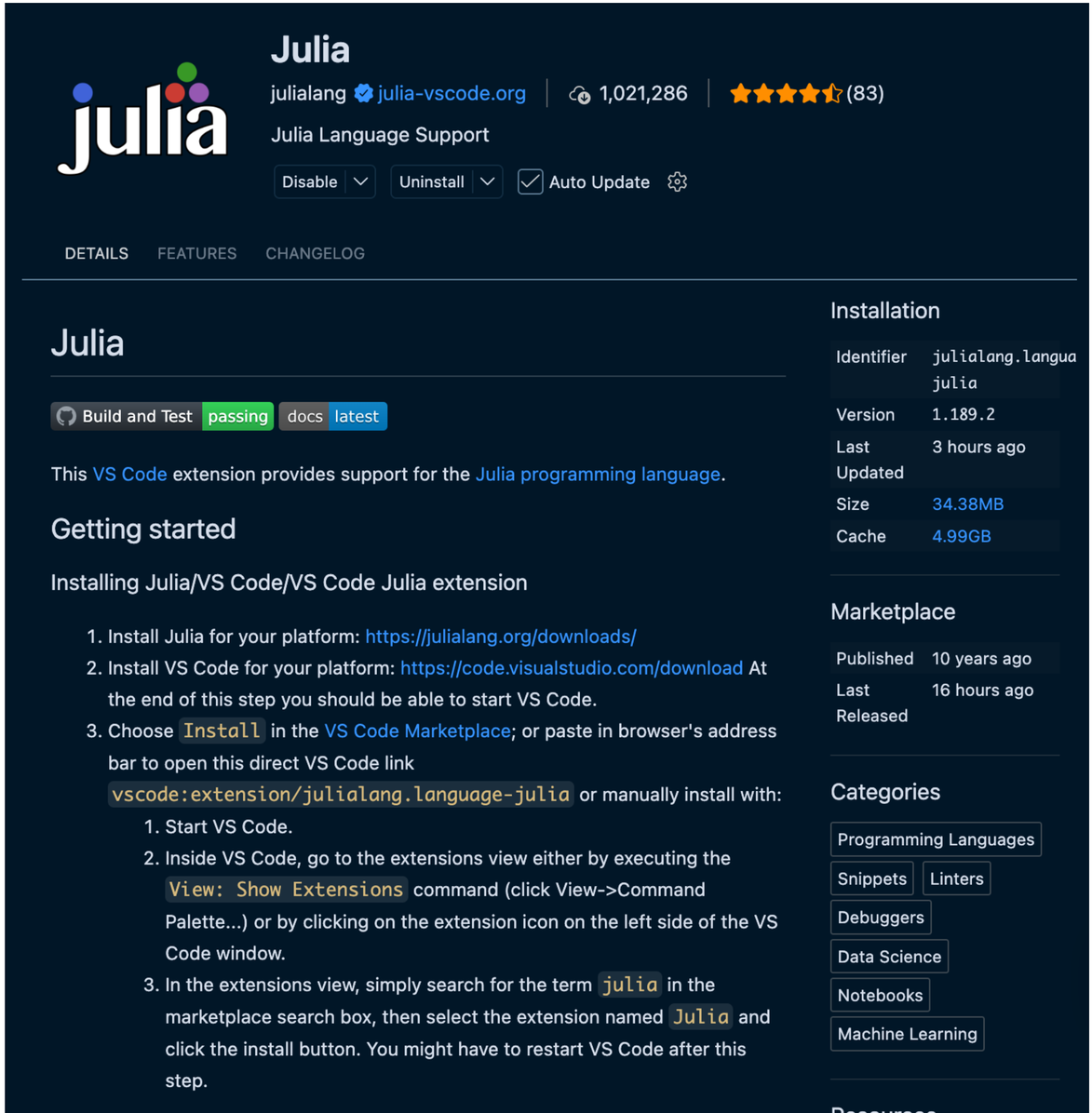
This will install the latest stable version of Julia, as well as the `juliaup` tool. Start Julia from the command-line by typing `julia`. See `juliaup --help` for how to configure installed versions.

If you prefer to use manual installation using a GUI-based installer, see the Manual Downloads page.

I run Julia in Visual Studio Code with the Julia extension. However, you can run Julia in the browser, like python.

Using Julia

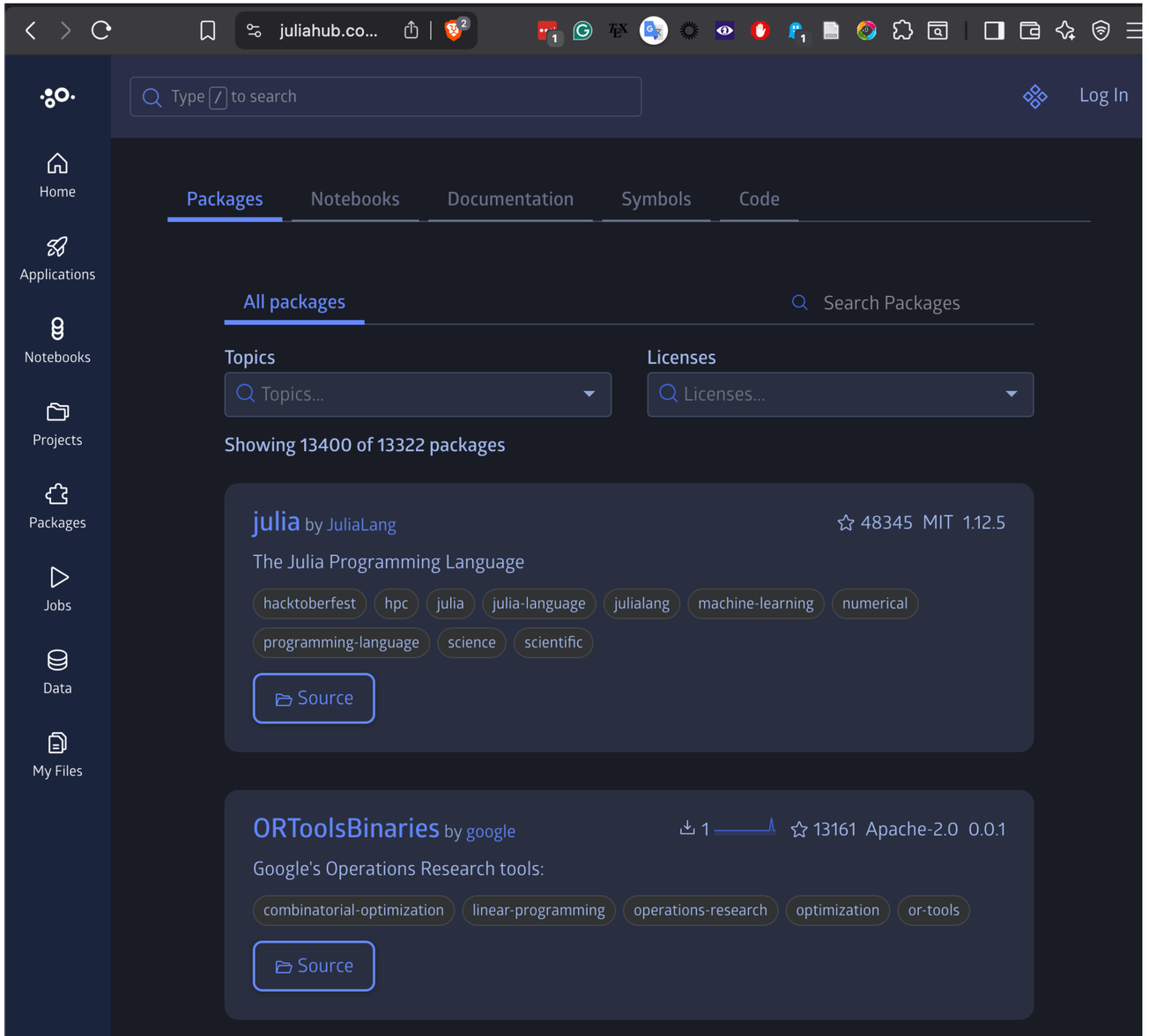
I recommend running Julia in Visual Studio Code with the Julia extension. However, you can run Julia in the browser, like Python.



Using Julia

Many basic mathematical operations are included in the base framework, including complex numbers.

For more sophisticated calculations, including **Linear Algebra**, **Machine Learning** and **Topological Data Analysis**, we use libraries. This can be found on JuliaHub.com



Using Julia

The package manager and local environments

Examples

Automatic Differentiation

Traditional finite difference methods can be noisy and expensive

- In lecture 1, you studied backpropagation, based on the chain rule. Here, we will consider forward differentiation.

- Given a function $f : \mathbb{R} \to \mathbb{R}$, we evaluate the derivative at a point x' by enlarging our number system to the dual numbers

$$\{x + y\epsilon \mid x, y \in \mathbb{R}\}$$

with

$$\epsilon^2 = 0$$

by which

$$f(x' + \epsilon) = f(x') + f'(x')\epsilon$$

Automatic Differentiation

For example, for the function

$$f(x) = ax^2 + bx + c$$

direct substitution yields

$$\begin{aligned} f(x' + \epsilon) &= a(x' + \epsilon)^2 + b(x' + \epsilon) + c \\ &= a(x'^2 + 2x'\epsilon + \epsilon^2) + b(x' + \epsilon) + c \\ &= (ax'^2 + bx' + c) + (2ax' + b)\epsilon \end{aligned}$$

yields the familiar result. This works for any programmatic operation in Julia

Automatic Differentiation

```julia
using ForwardDiff

f(x) = 5. * x^2 - 3. * x + 2.

ForwardDiff.derivative(f, 2.)
```

✓ 0.3s

```
17.0
```

Differential equations

One often encounters differential equations. To illustrate how Julia can be used to solve them, we consider the DifferentialEquations.jl library.

Consider the Harmonic Oscillator

$$\ddot{x}(t) + \omega^2 x(t) = 0$$

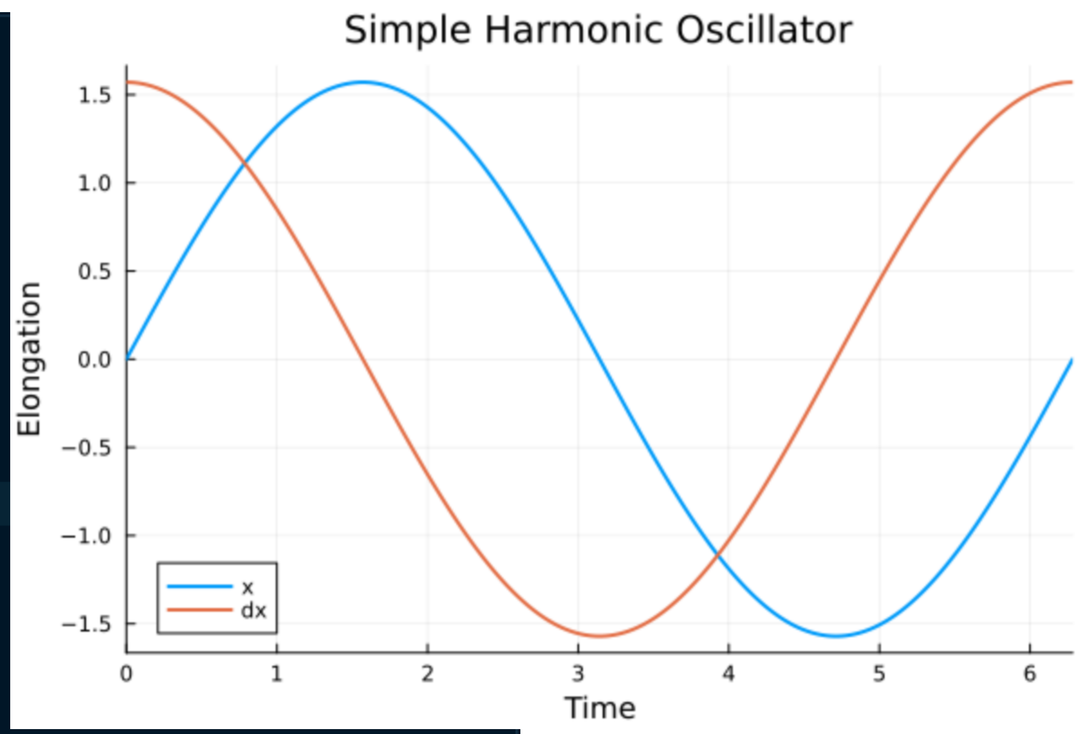
with the boundary conditions

$$x(0) = x_0\,, \quad \dot{x}(0) = v_0$$

Differential equations

```julia
# Simple Harmonic Oscillator Problem
import OrdinaryDiffEq as ODE, Plots

#Parameters
ω = 1

#Initial Conditions
x₀ = [0.0]
dx₀ = [π / 2]
tspan = (0.0, 2π)

#Define the problem
function harmonicoscillator(ddu, du, u, ω, t)
    ddu .= -ω^2 * u
end

#Pass to solvers
prob = ODE.SecondOrderODEProblem(harmonicoscillator, dx₀, x₀, tspan, ω)
sol = ODE.solve(prob, ODE.DPRKN6())

#Plot
Plots.plot(sol, idxs = [2, 1], linewidth = 2, title = "Simple Harmonic Oscillator", xaxis =
"Time", yaxis = "Elongation", label = ["x" "dx"])
```

✓ 23.9s Julia



Polynomial arithmetic

Polynomial equations are central to mathematics and physics. Consider for example the study of algebraic curves in algebraic geometry. Let us work over the ring of one-dimensional polynomialsusing the Polynomials.jl library. Let us add two polynomials and find all the roots of a polynomial.

```julia
using Polynomials
P₁ = Polynomial([1,0,3,4])
P₂ = Polynomial([4,-2,4])
@show P₁, P₂

P₃ = P₁ + P₂
```

✓ 0.1s Julia

```
(P₁, P₂) = (Polynomial(1 + 3*x^2 + 4*x^3), Polynomial(4 - 2*x + 4*x^2))
```

$5 - 2\cdot x + 7\cdot x^2 + 4\cdot x^3$

Polynomial arithmetic

Polynomial equations are central to mathematics and physics. Consider for example the study of algebraic curves in algebraic geometry. Let us work over the ring of one-dimensional polynomialsusing the Polynomials.jl library. Let us add two polynomials and find all the roots of a polynomial.

```julia
using Polynomials
P₁ = Polynomial([1,0,3,4])
roots(P₁)
```

✓ 0.0s Julia

```
3-element Vector{ComplexF64}:
                -1.0 + 0.0im
 0.12500000000000006 - 0.4841229182759272im
 0.12500000000000006 + 0.4841229182759272im
```

Homotopic Continuation

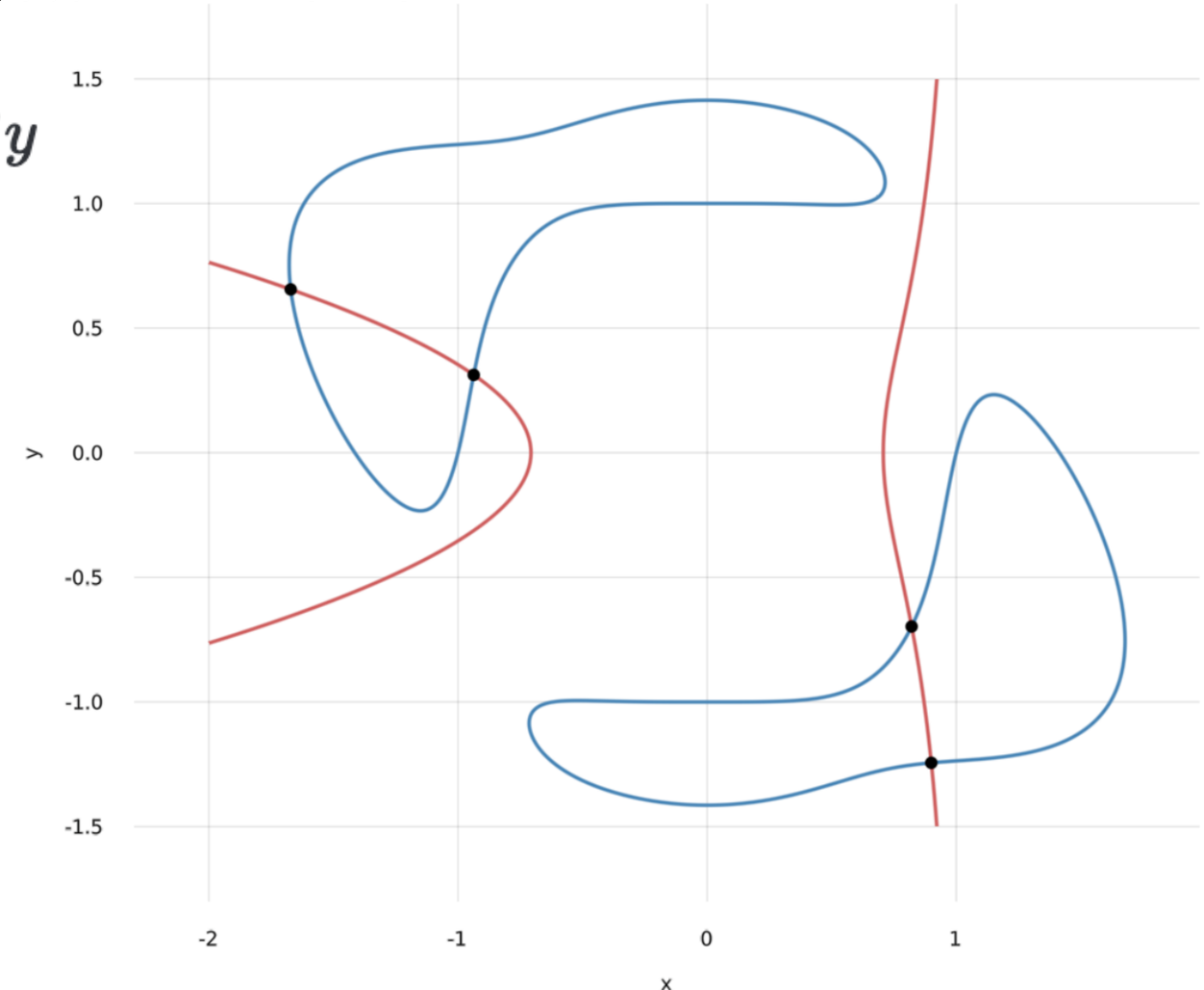
In algebraic geometry, we often consider the zero set of a set of multivariate polynomials over the complex numbers. Finding all zero sets can be very tricky with the traditional Newton-Raphson method. For these Homotopic Continuation can provide a great solution. Let us explore the library HomotopyContinuation.jl and find all roots of a set of two multivariate polynomials.

Homotopic Continuation

We want to evaluate the zero set $V(f_1, f_2)$ for the polynomials

$$f_1(x,y) = (x^4 + y^4 - 1)(x^2 + y^2 - 2) + x^5 y$$
$$f_2(x,y) = x^2 + 2xy^2 - 2y^2 - \frac{1}{2}$$

Starting from a set of equations
$G(x,y) = (g_1(x,y), g_2(x,y)) = 0$
for which we know the roots, we build the homotopy
$H(x,y;\lambda) = (1-\lambda)G(x,y) + \lambda F(x,y)$
and track the evolution of the roots as a function of λ



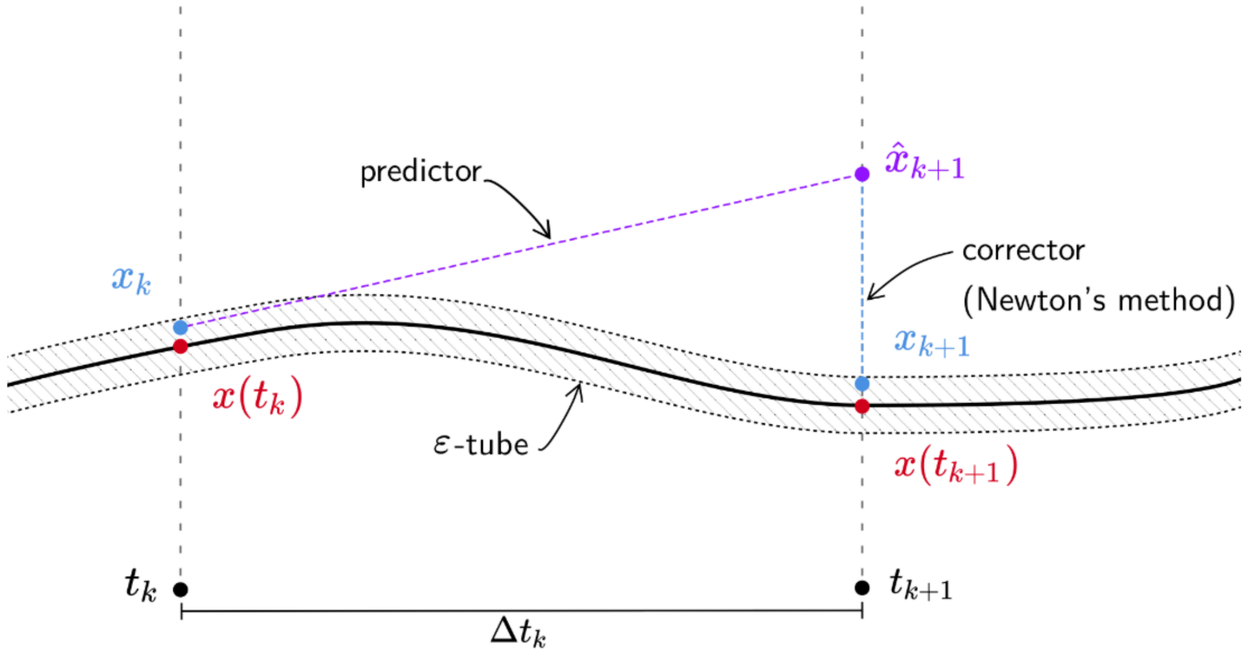
Homotopic Continuation

$$H(x, y; \lambda) = (1 - \lambda)G(x, y) + \lambda F(x, y)$$

Explicitly, we define a trajectory $\mathbf{x}(\lambda) = (x(\lambda), y(\lambda))$ satisfying the equation

$$H(\mathbf{x}(\lambda); \lambda) = 0$$

which we solve with an ODE. When G as at least as many roots as F, we are guaranteed to find all the roots of F



Homotopic Continuation

```julia
# load the package
using HomotopyContinuation
# declare variables x and y
@var x y
# define the polynomials
f₁ = (x^4 + y^4 - 1) * (x^2 + y^2 - 2) + x^5 * y
f₂ = x^2+2x*y^2 - 2y^2 - 1/2
F = System([f₁, f₂])
result = solve(F)
```

✓ 4.0s Julia

```
Result with 18 solutions
========================
• 18 paths tracked
• 18 non-singular solutions (4 real)
• random_seed: 0x8388d1bd
• start_system: :polyhedral
```

Homotopic Continuation

```julia
solutions(result)
```

✓ 1.2s Julia

```
18-element Vector{Vector{ComplexF64}}:
 [0.9443571312488813 - 0.311863501797266im, 0.32083818529039954 - 0.9677296009728283im]
 [1.0866676911062134 - 0.3290700669978767im, -0.24048106708661138 + 1.135021582399367im]
 [0.8708494909007278 - 0.03055030123533539im, 0.9716347867303132 - 0.21528324968394966im]
 [1.7132941375582666 + 0.5813863945698413im, 0.04751413478085412 - 1.252792951007688im]
 [-0.9368979667963299 - 6.018531076210112e-36im, 0.312284081738601 - 4.81482486096809e-35im]
 [0.8209788924342627 + 6.726232628759122e-44im, -0.6971326459489459 + 3.587324068671532e-43im]
 [1.7132941375582666 - 0.5813863945698412im, 0.04751413478085411 + 1.252792951007688im]
 [-1.0665536440076708 - 0.1424944286215267im, -0.3998429331325363 - 0.0787123814749347im]
 [0.8708494909007278 + 0.03055030123533539im, 0.9716347867303131 + 0.21528324968394968im]
 [1.0866676911062132 + 0.3290700669978766im, -0.2404810670866113 - 1.1350215823993668im]
 [0.8999179208471728 + 2.311115933264683e-33im, -1.2441827613422727 - 3.0814879110195774e-32im]
 [-1.6714213928380033 - 2.571393892423754e-39im, 0.6552051858720409 + 8.265194654219209e-40im]
 [0.0694425558897196 - 1.0734210145259255im, 0.2852544211796043 + 0.7076856100161802im]
 [0.06944255588971959 + 1.0734210145259255im, 0.2852544211796043 - 0.7076856100161802im]
 [0.07565391048031056 - 0.9487419814734106im, -0.24800445792173503 - 0.6838307098593375im]
 [0.07565391048031057 + 0.9487419814734106im, -0.24800445792173506 + 0.6838307098593375im]
 [0.9443571312488814 + 0.311863501797266im, 0.3208381852903996 + 0.9677296009728285im]
 [-1.0665536440076708 + 0.14249442862152673im, -0.39984293313253627 + 0.0787123814749347 1im]
```

Machine Learning

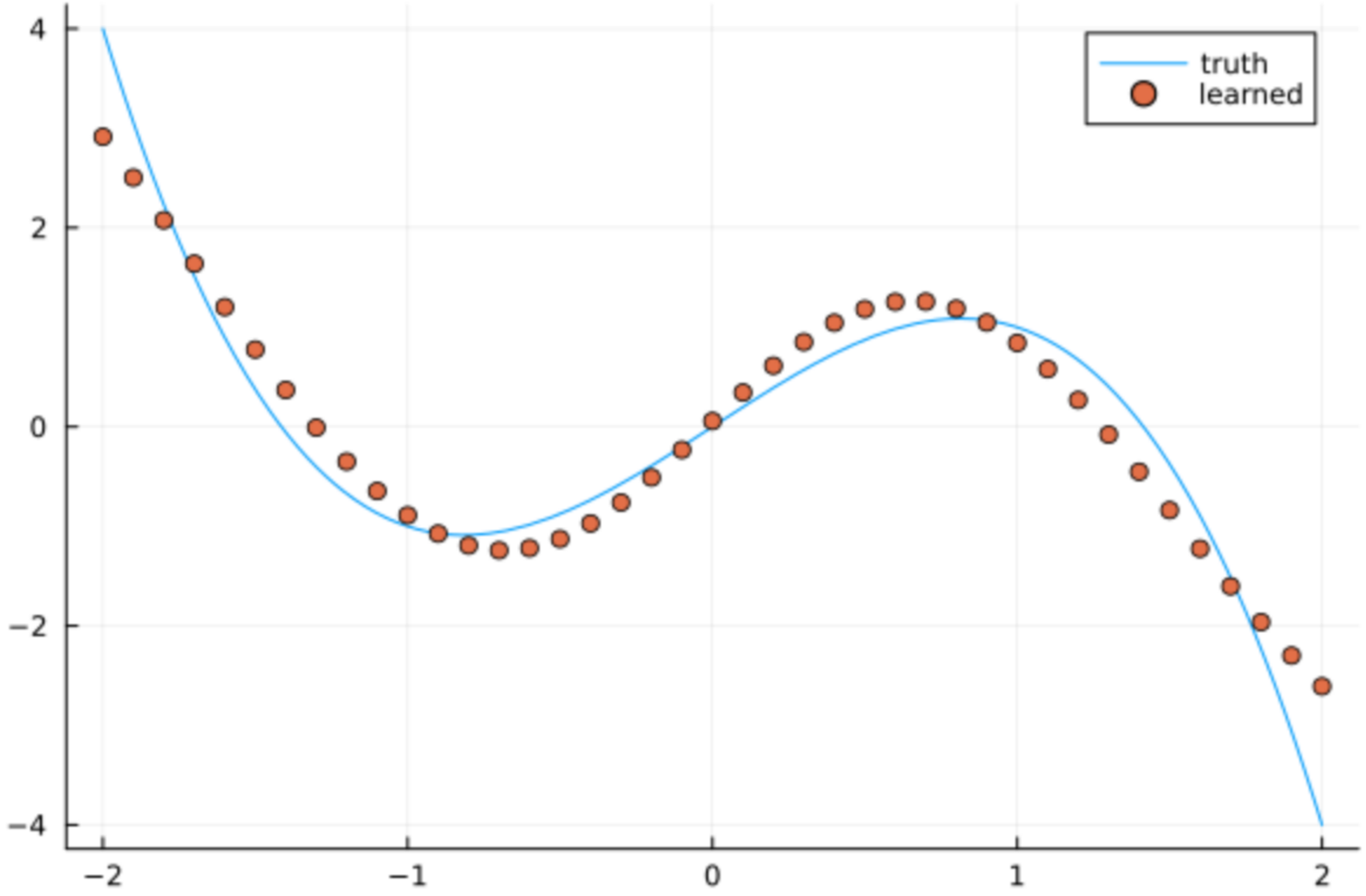
Julia has a fully capable Machine Learning library built in Julia, known as Flux.jl.

```julia
using Flux, Plots
data = [(x, 2x-x^3) for x in -2:0.1f0:2]

model = let
  w, b, v = (randn(Float32, 23) for _ in 1:3)  # parameters
  x -> sum(v .* tanh.(w*x .+ b))               # callable
end

opt_state = Flux.setup(Adam(), model)
for epoch in 1:100
  Flux.train!((m,x,y) -> (m(x) - y)^2, model, data, opt_state)
end

plot(x -> 2x-x^3, -2, 2, label="truth")
scatter!(model, -2:0.1f0:2, label="learned")
```

✓ 1.0s

Machine Learning

Julia has a fully capable Machine Learning library built in Julia, known as Flux.jl.



Going to the essentials

- Libraries are a great resource for research
- By its nature, we explore new concepts and ideas in scientific explorations
- It is useful to adapt existing libraries and create libraries from scratch
- When you can build calculations to their essence, you have a way to progress your science
- Sometimes, one can borrow from computer science. Sometimes, you have to invent your own.

Summary

- Julia is a new and quickly developing programming language that is currently being adopted in both academia and beyond

- With Just-In-Time compilations and instances, Julia is as readable as Python and as quick as C++ (when done right). The two-language problem.

- The libraries developed in Julia are a great resource for a vast ranging or problems in scientific computing

- Thank you for your attention!